

Rilevamento e contenimento di *lateral movement* in una LAN emulata tramite SDN

Studente: Alessandro Gragnaniello **Matricola:** [DE9000037]

Repository: https://github.com/giovannistanco/NCIS_unina/tree/main/2026/gragnaniello_alessandro

Insegnamento: Network and Cloud Infrastructures — Prof. Giorgio Ventre — A.A. 2025/2026

1 Obiettivo e scenario

Questo progetto nasce dalla volontà di voler rilevare un host malevolo su una rete locale domestica. A questo scopo bisogna definire cosa sia un "host malevolo", questo è un host che è stato compromesso all'interno della rete e che ora vuole "infettare" anche altri dispositivi, questo fenomeno si chiama *lateral movement*. Per arrivare a questo scopo si è deciso di realizzare un controller SDN che **rileva** e **isola** automaticamente un host che esegue una scansione orizzontale della propria sottorete, cioè la fase di ricognizione che precede il lateral movement in una LAN.

Al fine di rilevare questo host, gran parte del lavoro è stato speso per capire quale fosse la sua *firma di traffico*. Di fatto, in una rete locale in genere vi sono una serie di host che, di rado parlano tra di loro ma, molto spesso parlano con il router della rete per contattare internet ed essere contattati. Quindi la maggior parte del traffico in una rete locale è di tipo NORD-SUD, il traffico di uno scanner, invece, è di tipo EST-OVEST ed è questa l'anomalia che si vuole isolare nel progetto. Di fatto, la firma della scansione è: molti tentativi di contatto laterale, quasi tutti falliti, ciascuno di volume trascurabile.

Topologia: Per simulare questo scenario si è realizzata una LAN domestica emulata in Mininet: uno **switch OpenFlow** ('s1') e 13 host collegati a stella. MAC e IP sono assegnati esplicitamente e in modo coerente — l'host *i*-esimo ha MAC '00:00:00:00:00:i in esadecimale' e IP '10.0.0.i' — così che ogni riga di log sia leggibile senza tabelle di conversione.

I ruoli sono una convenzione dello scenario, non una configurazione del codice: 'h1' rappresenta il gateway domestico, 'h2'–'h12' i dispositivi normali (PC, TV, stampante, IoT), 'h13' l'host compromesso.

2 Il controller

Il controller è realizzato tramite un singolo modulo python (`detector.py` nel repo) che deriva da `simple_switch_13.py` della distribuzione Ryu. Il codice originale apprende le associazioni MAC-porta e installa regole di inoltro; tutto ciò che segue è aggiunto.

Lo scopo del controller è **rilevare** l'host malevolo (tramite firma di traffico) e **isolarlo**.

2.1 Cambiamenti rispetto `simple_switch_13`

Le aggiunte sono tre strutture dati e tre metodi. Ciascuna è motivata nelle sottosezioni seguenti, nell'ordine in cui interviene: osservazione, decisione, azione.

2.2 Le tre feature e i due canali di osservazione

Innanzitutto definiamo la firma del traffico dell'host malevolo. Il nostro scenario è caratterizzato da una rete /24: 254 indirizzi assegnabili, lo scopo dello scanner è scoprire *quali indirizzi sono attivi* e qual'è il loro indirizzo MAC. Lo scanner lancia un **ping** verso ogni indirizzo, ma verso i 241 indirizzi

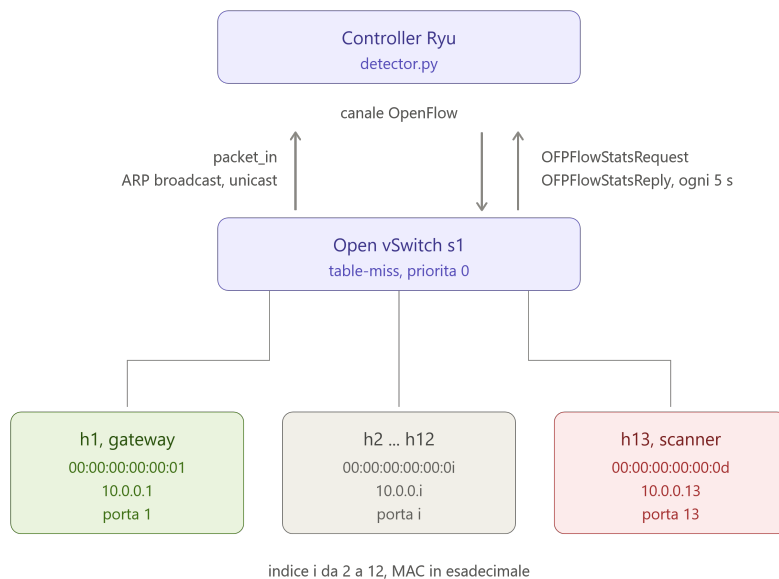


Figura 1: Topologia a stella: 13 host su un singolo switch OpenFlow.

vuoti l'**ICMP Echo Request non lascia mai la scheda**: lo stack lo trattiene in attesa di una risoluzione ARP che non arriver . Ci  che finisce sul filo   quasi esclusivamente ARP, e verso ogni indirizzo vuoto lo stack **ritrasmette tre volte** prima di arrendersi.

Packet_in : Ora, lo switch inizialmente ha installata una sola *flow rule*, la *table-miss* a priorit  0; Un frame che non trova corrispondenza in nessuna entry a priorit  superiore risale quindi al controller in un messaggio **packet_in** (OFPT_PACKET_IN, il messaggio OpenFlow con cui lo switch consegna il frame al piano di controllo).

Ci  significa che ogni ARP request inviata da un host in broadcast deve necessariamente passare per il controller. Quest    la prima metrica che ci interessa.

Sempre dai **pkt_in** possiamo ricavare un altro dato importante. Quando la destinazione di un pkt   *unicast*, possiamo capire anche quali e, quante, destinazioni vengono toccate dallo scanner in una certa finestra. Per lo scanner, nella nostra rete di 13 host, ci saranno 12 echo request che andranno a buon fine e, costituiscono il **fan-out** dello scanner. Il fan-out di uno scanner misura quindi **quanti host rispondono, non quanti ne sonda**:   una grandezza della rete, non dell'attaccante. Se ne terr  conto in §2.3.

Polling delle flow stats : Infine possiamo effettuare un **polling** delle flow stats sullo switch per capire anche la *profondit * del traffico ossia, per flusso, quanti pacchetti sono stati inviati. Questo viene fatto tramite un *thread* che ogni 5 secondi interroga gli switch registrati tramite **OFPFlowStatsRequest**.

I dati nella tabella sono stati ottenuti sperimentalmente facendo fare dei ping di 20 pkt agli host legittimi e una scansione a quello malevolo.

Il segnale forte   il fallimento, non la larghezza di banda. Un host legittimo interroga indirizzi che esistono: la sua ARP Request ottiene risposta, la Reply unicast fa installare una flow entry, e da quel momento il dialogo non risale mai pi  al controller. Lo scanner interroga 241 indirizzi vuoti: nessuna risposta, nessuna regola installata, e ogni tentativo ricomincia da capo. Un indirizzo vuoto non costa un 'pkt_in': ne costa tre.

Aggiunta	Tipo	Ruolo
<code>flow_prev</code>	dict	Contiene l'ultima lettura del <code>pkt_count</code> per ogni entry di inoltro attualmente viva sullo switch. Serve a calcolare il delta fra due letture che viene salvato in <code>window</code>
<code>window</code>	dict	Contiene <i>per ogni host</i> le <i>feature</i> rilevate in una finestra di 5 secondi e viene azzerata ciclicamente. E' aggregata per host perchè noi dovremo isolare il singolo host non un flusso.
<code>flagged</code>	dict	Contiene gli host che sono stati messi in quarantena e l'istante in cui saranno "liberati".
<code>_monitor</code>	metodo	Manda la richiesta di FlowStats agli switch
<code>_evaluate</code>	metodo	Applica il criterio a <code>window</code> , host per host, saltando quelli già in quarantena; se scatta, registra l'host in <code>flagged</code> e chiama <code>_contain</code> .
<code>_contain</code>	metodo	installa la regola di blocco

Tabella 1: Aggiunte rispetto al *learning switch* originale.

#	Feature	Baseline	Scanner	Separazione
1	ARP Request in broadcast	1	735	$\times 735$
3	Fan-out	2	12	$\times 6$
2	Volume aggregato	11	0-12	nessuna

Tabella 2: Le tre feature.

Conseguenza strutturale: lo scanner è quasi invisibile nelle statistiche di flusso, perché 241 tentativi su 253 non producono alcuna flow entry. Il polling non scopre host, aggiunge un attributo a host già noti dai 'pkt_in'.

2.3 Criterio di rilevamento:

Le feature sono aggregate per host sorgente su finestre di 5 secondi. Il criterio è deliberatamente una sola condizione necessaria:

```
if f['arp'] >= self.ARP_MIN:                                # ARP_MIN = 20
    self.flagged[src] = time.time() + self.CONTAIN_SEC
    sev = 'HIGH' if len(f['dsts']) >= self.FANOUT_MIN else 'LOW'
    self.logger.warning("ALERT_%s_%s_arp=%s_fanout=%s_pkts=%s", ...)
    self._contain(src)
```

'ARP_MIN = 20' è 20 volte il massimo di baseline misurato.

Il fan-out non entra in AND, ed è una decisione verificata sul campo. Il fan-out di uno scanner è pari al numero di host che rispondono, non a quelli sondati. In una sessione di prova è stata osservata una finestra con 'arp=137, fanout=0' — un AND avrebbe mancato il rilevamento. Il fan-out sopravvive quindi come qualificatore di severità ('HIGH'/'LOW'): conferma il rilevamento quando c'è, non lo nega quando manca.

3 Risultati sperimentali

I risultati provengono da sessioni di laboratorio registrate con `script`; gli estratti seguenti sono tutti tratti dalla stessa sessione, salvo dove indicato.

Traffico legittimo:

Un ping fra due host non allerta il controller e installa due regole di inoltrato, una per verso:

```
mininet> h2 ping -c 20 -i 1 -n 10.0.0.1
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
  n_packets=21, n_bytes=2002, priority=1,in_port="s1-eth1",
    dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=output:"s1-eth2"
  n_packets=20, n_bytes=1904, priority=1,in_port="s1-eth2",
    dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
  n_packets=120, n_bytes=7000, priority=0 actions=CONTROLLER:65535
```

I due contatori sono **asimmetrici**: 21 pacchetti nel verso $h1 \rightarrow h2$ e 20 in quello opposto. Il primo Echo Request viene inoltrato dal controller con un `packet_out`, che non attraversa la tabella e quindi non è contato dalla regola che esso stesso ha appena creato. A rete ferma il ciclo di polling stampa ogni 5 s una riga per host, con le tre feature a zero, e **stats reply: 1 entry** — la sola *table-miss*.

Scansione: rilevamento

```
mininet> h13 bash -c 'for i in $(seq 1 254); do ping -c1 -W1 10.0.0.$i & done'
```

Il comando lancia in parallelo un ping verso ogni indirizzo assegnabile (`-c1`: un solo tentativo; `-W1`: un secondo di attesa). Il controller reagisce entro una finestra:

```
window 00:00:00:00:00:0d arp=735 fanout=12 pkts=0
ALERT HIGH 00:00:00:00:00:0d arp=735 fanout=12 pkts=0
CONTAIN 00:00:00:00:00:0d dpid=1 prio=100 hard_timeout=30 s
window 00:00:00:00:00:02 arp=0 fanout=1 pkts=0
window 00:00:00:00:00:03 arp=0 fanout=1 pkts=0
[... le altre 10 righe, tutte con arp=0 ...]
stats reply: 26 entry
```

L'ALERT è **uno solo** e sull'host corretto; i dodici host legittimi restano a `arp=0` nella stessa finestra. Da questo ciclo in poi **h13 scompare dall'elenco**: `_evaluate` salta gli host in quarantena, e soprattutto i suoi pacchetti non arrivano più al controller.

Contenimento e riabilitazione

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1 | grep -E "priority=100|priority=0 "
  duration=5.700s, n_packets=36, n_bytes=1512, hard_timeout=30,
  priority=100,dl_src=00:00:00:00:00:0d actions=drop
  duration=42.663s, n_packets=858, priority=0 actions=CONTROLLER:65535

mininet> py time.sleep(35)
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1 | grep -E "priority=100|priority=0 "
  duration=99.619s, n_packets=870, priority=0 actions=CONTROLLER:65535
```

La regola esiste, **agisce** e **scade**: nel secondo dump resta la sola *table-miss*.

Lo stesso ciclo di vita è osservabile dal solo log del controller, tramite il numero di entry restituite a ogni polling:

Fase	Entry lette
rete a riposo	1 (sola <i>table-miss</i>)
dopo la scansione, <i>drop</i> attiva	26
dopo la scadenza	25

Il passaggio da 26 a 25 avviene **un ciclo di polling dopo** la riga **quarantena scaduta**: il controller libera l'host prima che la regola sparisca dallo switch. È il disallineamento fra le due metà della riabilitazione, discusso in §5.

4 Riproduzione dell'esperimento

I test sono stati eseguiti nel seguente ambiente: Ubuntu 22.04 ARM64, Python 3.10.12, Mininet 2.3.0, Open vSwitch 2.17.12, Ryu 4.34, OpenFlow 1.3.

Prerequisito obbligatorio. Ryu 4.34 non è compatibile con le versioni di `eventlet` disponibili per Python 3.10 e **non si avvia**. Il repository include `ryu-eventlet-patch.diff`, che sostituisce il sentinella `ALREADY_HANDLED` in `ryu/app/wsgi.py` con una classe locale equivalente. Va applicato dopo l'installazione delle dipendenze (`ryu-requirements.txt`) e prima del primo avvio.

Avvio. Il controller va lanciato **prima** della rete, in due terminali distinti:

```
# terminale 1
source ~/ryu-venv/bin/activate && ryu-manager detector.py
# terminale 2
sudo mn -c && sudo python3 topologia/casa.py
```

Traffico legittimo: Dalla CLI di Mininet, due host pingano il gateway:

```
h2 ping -c 20 -i 1 -n 10.0.0.1 &
h5 ping -c 20 -i 1 -n 10.0.0.1 &
```

Atteso: righe `window` con `arp` pari a 0 o 1 per gli host attivi, nessun **ALERT**.

Scansione e contenimento. In questa fase v'è lanciato il comando che fa eseguire la scansione all'host `h13`. È importante eseguire il `dump-flows` entro 30 secondi dal `ping`, altrimenti la regola di `drop` scadrebbe.

```
sh date
h13 bash -c 'for i in $(seq 1 254); do ping -c1 -W1 10.0.0.$i & done'
sh ovs-ofctl -O OpenFlow13 dump-flows s1 | grep -E "priority=100|priority=0 "
py time.sleep(35)
sh ovs-ofctl -O OpenFlow13 dump-flows s1 | grep -E "priority=100|priority=0 "
sh date
```

Dopo lo `sleep`, eseguendo di nuovo `dump-flows` si può osservare che la regola scade correttamente.

Risultati osservabili: Dall'esecuzione si possono osservare alcuni eventi ricorrenti, il valore dei conteggi dipende dalla particolare esecuzione.

1. nel log del controller, una riga **ALERT** con `arp` di due ordini di grandezza sopra la soglia, seguita immediatamente da **CONTAIN**;
2. nel primo `dump-flows`, una entry `priority=100 ... actions=drop` con il MAC di `h13` e `hard_timeout=30`;
3. il conteggio `stats reply` che sale a 26 entry;
4. nel secondo `dump-flows`, l'assenza di quella entry;
5. nel log, la riga **quarantena scaduta**.

Nota metodologica. Un `pingall` eseguito prima della misura ne invaliderebbe i risultati: pre-installa tutte le regole di inoltro e azzeri il traffico diretto al controller, cioè proprio il canale su cui si basano le prime due feature.

5 Limiti noti e sviluppi

In quest'ultima sezione vengono riportati alcuni limiti noti e sviluppi possibili.

- **MAC spoofing.** Rilevamento e contenimento si basano entrambi su `eth_src`. Uno scanner che *randomizzi* il *MAC* a ogni richiesta frammenterebbe l'aggregazione, portando ciascun MAC fittizio sotto soglia di rilevazione.
- **Densità della sottorete.** In una rete molto popolata le ritrasmissioni verso gli indirizzi vuoti scompaiono e il conteggio si riduce di circa tre volte (nell'ordine di 260 anziché 771). Il segnale è attenuato ma resta ampiamente sopra soglia: il rilevamento continuerebbe a funzionare.
- **Contenimento reattivo.** Il sistema non impedisce che la scansione avvenga: la *termina*, con una latenza dalla finestra di 5 s. Ciò che previene è il movimento laterale *verso* gli host già scoperti.

Sviluppi. Per verificare la robustezza si era pensato di provare lo stesso controller su una topologia a due switch: il traffico di h13 presenterebbe coppie (`dpid`, `in_port`) diverse, ma il rilevamento resterebbe valido perché l'aggregazione è per `eth_src` e `_contain` installa la regola su tutti gli switch registrati — il tutto **senza modificare una riga di codice**. In modo simmetrico, si era pensato di sostituire il *ping sweep* con `nmap -sn`, cambiando cioè l'attacco e lasciando invariato il rilevatore.
